

Streamlit + Plotly para dashboards biomédicos

Tutorial aplicado: monitoreo neuromuscular post-ictus mediante sEMG

Señales Bioeléctricas y Visualización Interactiva

2026-09-16

Propósito del tutorial

Este tutorial no empieza en el código. Empieza en una necesidad clínica y termina en una aplicación modular.

Al finalizar, el estudiante deberá poder:

- describir las partes de un dashboard biomédico;
- transformar una pregunta clínica en indicadores de sEMG;
- diseñar un pipeline reproducible de análisis;
- implementar un dashboard multi-script con Streamlit y Plotly;
- interpretar los resultados sin convertir el dashboard en diagnóstico automático.

Caso de trabajo

Solicitud inicial del médico

Necesito una herramienta para revisar señales de sEMG en personas con antecedente de ictus. Quiero inspeccionar la calidad de la señal, observar activación muscular, comparar indicadores entre canales y preparar un esquema de seguimiento funcional. No necesito un clasificador automático; necesito evidencia visual y cuantitativa para discusión clínica.

La respuesta de ingeniería debe traducir esta solicitud a datos, procesamiento, indicadores y visualizaciones.

Advertencia conceptual

Error que debe evitarse

No se debe afirmar que la sEMG mide directamente la evolución del ictus.

La sEMG permite observar cambios en actividad eléctrica muscular superficial. La evolución clínica del ictus se valora con escalas, examen neurológico, funcionalidad, imagen médica y seguimiento médico.

En este tutorial hablaremos de **seguimiento neuromuscular post-ictus**, no de diagnóstico ni pronóstico automático.

Sección 1

Parte I - Antes del código: qué es un dashboard biomédico

Dashboard: definición funcional

Un dashboard biomédico es un sistema de visualización que integra:

- ① datos fisiológicos;
- ② metadatos clínicos;
- ③ procesamiento reproducible;
- ④ indicadores cuantitativos;
- ⑤ visualizaciones interactivas;
- ⑥ advertencias de interpretación.

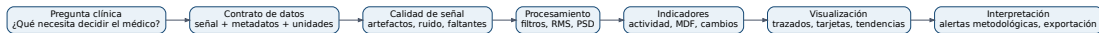
No es simplemente una colección de gráficas.

Dashboard frente a reporte estático

Elemento	Reporte	Dashboard
Interacción	Baja	Alta
Exploración temporal	Limitada	Navegable
Reprocesamiento	Manual	Controlado por parámetros
Inspección de señal	Fija	Selectiva
Usuario principal	Lector	Analista o clínico

En sEMG, el dashboard es útil porque la señal cambia con el tiempo, el canal, la tarea y la calidad de adquisición.

Partes de un dashboard biomédico



La interfaz visible es solo la última capa. Si el contrato de datos o el procesamiento están mal definidos, el dashboard será visualmente atractivo pero técnicamente débil.

Componentes mínimos del dashboard sEMG

Componentes visibles

- selector de sujeto o registro;
- metadatos clínicos;
- visor de señal cruda;
- visor de señal filtrada;
- panel de indicadores;
- gráficas de tendencia.

Componentes no visibles

- carga parcial del registro;
- validación de canales;
- control de calidad;
- filtrado;
- cálculo de indicadores;
- caché y manejo de errores.

Preguntas que debe responder

El dashboard debe responder preguntas concretas:

- ¿El segmento seleccionado tiene calidad suficiente?
- ¿Qué canales muestran mayor actividad eléctrica?
- ¿La actividad es continua, episódica o ausente?
- ¿Cómo se distribuye la potencia espectral?
- ¿Los indicadores cambian entre visitas o condiciones?
- ¿Qué limitaciones impiden interpretar clínicamente el resultado?

Si no hay preguntas explícitas, la visualización queda sin criterio técnico.

Sección 2

Parte II - Problema biomédico: ictus y sEMG

Del ictus a la actividad muscular

El ictus puede alterar vías motoras descendentes y modificar el control voluntario del músculo.

En términos funcionales pueden observarse:

- debilidad muscular;
- activación tardía o insuficiente;
- pérdida de selectividad muscular;
- compensaciones motoras;
- cambios en coordinación intermuscular.

La sEMG registra actividad eléctrica muscular superficial, no fuerza mecánica.

Qué observa la sEMG

La señal sEMG es la suma interferencial de potenciales de acción de unidades motoras detectados en superficie.

Por tanto, está afectada por:

- reclutamiento de unidades motoras;
- frecuencia de descarga;
- posición de electrodos;
- impedancia de piel;
- tejido subcutáneo;
- movimiento;
- ruido eléctrico.

Esto obliga a separar indicadores fisiológicos de indicadores de calidad.

Dataset base

El tutorial usa **Cerebral Vasoregulation in Elderly with Stroke (CVES)**, alojado en PhysioNet (Novak 2018).

Se utiliza porque permite trabajar con:

- grupo con antecedente de ictus;
- grupo control;
- metadatos clínicos;
- registros fisiológicos en formato WFDB;
- canales identificados como EMG.

Límite del dataset

CVES no prueba evolución longitudinal

CVES es adecuado para enseñar carga, procesamiento y visualización. Pero si se usa un solo registro por sujeto, no permite demostrar recuperación longitudinal.

El módulo de seguimiento del dashboard debe tratarse como estructura metodológica o simulación didáctica, no como evidencia clínica real.

Variables clínicas externas

Variable	Qué significa	Uso correcto
NIHSS	Severidad neurológica del ictus	Contexto clínico
mRS/MRS	Discapacidad funcional global	Contexto funcional
Stroke Side	Lado o hemisferio reportado	Interpretar con cautela
Symptoms	Descripción clínica	Lectura cualitativa

Estas variables no se calculan desde sEMG. Sirven para contextualizar la señal.

Traducción clínica a ingeniería

La solicitud médica se convierte en una cadena de decisiones:



Sección 3

Parte III - Indicadores desde la fisiología hasta la ingeniería

Selección de indicadores

No todos los indicadores son apropiados para cualquier pregunta.

En este tutorial se usan seis indicadores:

- RMS activo mediano;
- porcentaje de tiempo activo;
- episodios de activación por minuto;
- frecuencia mediana, MDF;
- cambio relativo del RMS activo;
- distancia relativa de MDF frente a referencia.

Indicador 1: RMS local

La señal filtrada se denota como:

$$x_f[n]$$

El RMS local en una ventana de N_w muestras es:

$$RMS[n] = \sqrt{\frac{1}{N_w} \sum_{i=n-N_w+1}^n x_f^2[i]}$$

Interpretación: magnitud local de activación eléctrica.

RMS activo mediano

Primero se define un umbral robusto T . Luego se calcula:

$$RMS_{act} = \text{mediana}\{RMS[n] \mid RMS[n] > T\}$$

Esto evita que los periodos de reposo dominen el indicador.

Uso prudente: comparar segmentos bajo protocolos equivalentes, no inferir fuerza absoluta.

Indicador 2: porcentaje de tiempo activo

Se define una máscara binaria:

$$a[n] = \begin{cases} 1, & RMS[n] > T \\ 0, & RMS[n] \leq T \end{cases}$$

Entonces:

$$\%Activo = 100 \frac{\sum_n a[n]}{N}$$

Interpretación: proporción del intervalo donde el músculo muestra actividad detectable.

Indicador 3: episodios por minuto

Un episodio es una secuencia continua donde $a[n] = 1$.

$$Episodios/min = \frac{N_{episodios}}{duracion\ en\ minutos}$$

Interpretación: fragmentación o frecuencia de activaciones musculares.

Debe acompañarse con duración mediana de episodios para evitar conclusiones incompletas.

Indicador 4: MDF mediante Welch

La densidad espectral de potencia se estima con Welch (?):

$$\hat{S}_{xx}(f) = \frac{1}{M} \sum_{m=1}^M P_m(f)$$

La frecuencia mediana *MDF* satisface:

$$\int_{f_L}^{MDF} P(f)df = \frac{1}{2} \int_{f_L}^{f_H} P(f)df$$

MDF resume cómo se reparte la potencia espectral.

Indicador 5: cambio longitudinal del RMS

Para una visita k , frente a la visita basal 0:

$$\Delta RMS_{rel,k} = 100 \frac{RMS_{act,k} - RMS_{act,0}}{RMS_{act,0}}$$

Interpretación: cambio porcentual de la activación eléctrica durante tareas equivalentes.

Condición: mismo músculo, tarea, ubicación de electrodos y procesamiento.

Indicador 6: distancia relativa de MDF

Cuando existe referencia contralateral o basal válida:

$$D_{MDF,k} = 100 \frac{|MDF_{paretico,k} - MDF_{ref,k}|}{MDF_{ref,k}}$$

Interpretación: distancia espectral relativa.

No implica automáticamente recuperación; solo muestra convergencia o divergencia espectral frente a una referencia definida.

Matriz fisiología - indicador - visualización

Fenómeno esperado	Indicador	Visualización
Activación muscular reducida	RMS activo	Barras por canal
Activación intermitente	Tiempo activo	Tarjeta + serie temporal
Fragmentación de contracción	Episodios/min	Tabla + barras
Cambios espectrales	MDF	PSD + tarjeta
Seguimiento funcional	ΔRMS_{rel}	Línea por visita
Diferencia con referencia	D_{MDF}	Tendencia o barra

El indicador determina la figura. No debe ocurrir al revés.

Sección 4

Parte IV - Pipeline teórico de desarrollo

Pipeline general

El desarrollo no se organiza por archivos. Se organiza por decisiones:

- ① requerimiento clínico;
- ② contrato de datos;
- ③ control de calidad;
- ④ preprocesamiento;
- ⑤ extracción de indicadores;
- ⑥ diseño visual;
- ⑦ validación;
- ⑧ implementación multi-script.

Los scripts aparecen después como consecuencia de esta estructura.

Etapas 1: requerimiento clínico

Se debe escribir una pregunta operacional:

Pregunta operacional

En un segmento de sEMG de un participante con antecedente de ictus, ¿qué canales muestran actividad detectable, cuál es su distribución espectral y cómo se podrían comparar estos indicadores en visitas repetidas bajo un protocolo estandarizado?

Esta pregunta define las métricas necesarias.

Etapa 2: contrato de datos

El contrato de datos especifica lo que cada función puede esperar.

Elemento	Requisito
Señal	matriz $N \times C$
Frecuencia	Hz, escalar positivo
Canales	nombres sin asumir músculo
Metadatos	sujeto, grupo, edad, NIHSS, mRS
Intervalo	inicio y duración en segundos
Unidades	microvoltios si el registro lo permite

Sin contrato de datos, la aplicación se vuelve frágil.

Etapa 3: calidad de señal

Antes de calcular indicadores se verifican problemas:

- datos faltantes;
- saturación o valores no finitos;
- exceso de baja frecuencia;
- contaminación de red eléctrica;
- segmento sin actividad útil;
- canales inexistentes.

La calidad no es decoración; es parte del resultado biomédico.

Etapa 4: preprocesamiento

Para sEMG superficial se aplica típicamente:

$x[n] \rightarrow$ interpolacion de faltantes $\rightarrow$ pasabanda $\rightarrow$ rectificacion $\rightarrow$ envolvente

En este tutorial:

- pasabanda aproximado: 20-450 Hz;
- notch de 60 Hz opcional;
- RMS local de 250 ms;
- envolvente lineal con pasabajos de 5 Hz.

Etapas 5: indicadores

Los indicadores se calculan después de calidad y filtrado.

Temporales

- RMS activo;
- tiempo activo;
- episodios/min;
- duración de episodios.

Espectrales y seguimiento

- PSD Welch;
- MDF;
- cambio relativo RMS;
- distancia relativa MDF.

Etapla 6: diseño visual

Cada visualización debe tener una función analítica.

Pregunta	Visualización
¿Hay artefactos?	señal cruda y filtrada
¿Dónde hay actividad?	RMS + umbral
¿Cómo se distribuye la potencia?	PSD Welch
¿Qué canal domina?	barras de indicadores
¿Hay cambio entre visitas?	líneas longitudinales

Etapla 7: validación

El dashboard debe validarse con pruebas mínimas:

- el filtro no debe fallar por frecuencia de muestreo;
- Welch debe recibir segmentos suficientemente largos;
- los indicadores deben manejar canales sin actividad;
- el dashboard no debe inventar lateralidad;
- las gráficas deben tener unidades y títulos;
- las advertencias metodológicas deben ser visibles.

Sección 5

Parte V - Streamlit y Plotly desde el diseño

Qué aporta Streamlit

Streamlit permite construir una aplicación desde scripts Python.

Su modelo es simple:

- 1 el usuario cambia un widget;
- 2 el script se ejecuta nuevamente;
- 3 los datos pesados se cachean;
- 4 la interfaz se reconstruye.

Esto obliga a separar funciones puras de elementos de interfaz.

Qué aporta Plotly

Plotly produce figuras interactivas basadas en:

- trazas de datos;
- layout;
- ejes;
- hover;
- leyendas;
- zoom y selección.

Para sEMG es útil porque el médico o estudiante puede inspeccionar segmentos, comparar canales y explorar PSD sin regenerar figuras manualmente.

Principio de separación

Regla de arquitectura

El procesamiento de señal no debe estar mezclado con el código de interfaz.

Incorrecto:

```
# dentro de la página Streamlit
filtered = signal.butter(...)
st.plotly_chart(...)
```

Correcto:

```
# processing.py
filtered = bandpass_savg(raw, fs)

# page.py
st.plotly_chart(raw_filtered_figure(...))
```

Por qué multi-script

Un dashboard multi-script permite:

- probar funciones por separado;
- reutilizar procesamiento;
- aislar errores;
- mantener páginas independientes;
- escalar el proyecto;
- evitar notebooks extensos y difíciles de auditar.

En docencia, también obliga a distinguir teoría, datos, procesamiento y visualización.

Sección 6

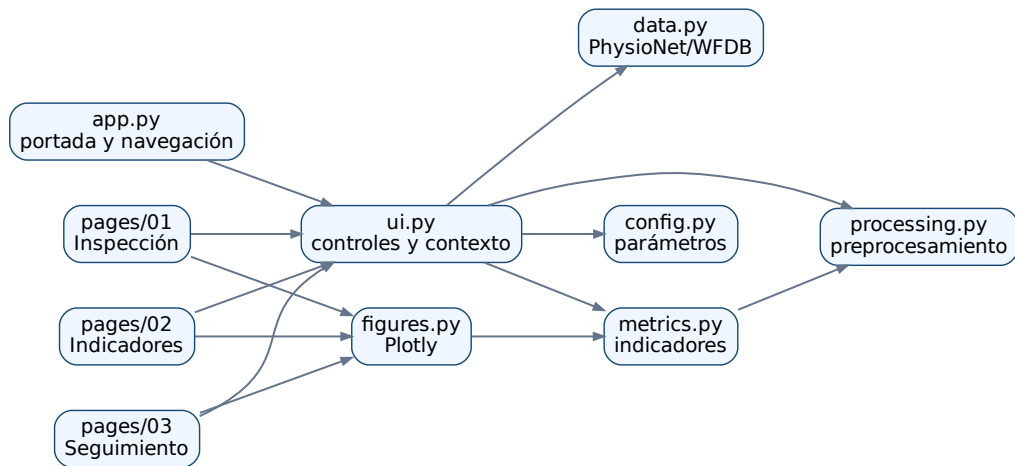
Parte VI - Arquitectura multi-script

Árbol del proyecto

```
semg_dashboard_project/  
  app.py  
  pages/  
    01_Inspeccion_sEMG.py  
    02_Indicadores.py  
    03_Seguimiento.py  
  src/semg_dashboard/  
    config.py  
    data.py  
    processing.py  
    metrics.py  
    figures.py  
    ui.py  
  pyproject.toml
```

Esta estructura separa entrada, páginas y lógica de dominio.

Dependencias entre módulos



Responsabilidad de cada módulo

Archivo	Responsabilidad principal
<code>config.py</code>	constantes, registros y límites
<code>data.py</code>	carga de PhysioNet/WFDB y metadatos
<code>processing.py</code>	filtros, RMS y envolventes
<code>metrics.py</code>	indicadores cuantitativos
<code>figures.py</code>	figuras Plotly
<code>ui.py</code>	controles, contexto y composición
<code>pages/*.py</code>	pantallas del dashboard
<code>app.py</code>	portada y configuración global

Regla de diseño de módulos

Cada archivo responde a una pregunta:

- `data.py`: ¿de dónde vienen los datos?
- `processing.py`: ¿cómo se transforma la señal?
- `metrics.py`: ¿qué se mide?
- `figures.py`: ¿cómo se visualiza?
- `ui.py`: ¿cómo interactúa el usuario?
- `pages/`: ¿qué flujo de análisis se presenta?

Si una función no responde a la pregunta del archivo, está mal ubicada.

Sección 7

Parte VII - Implementación como consecuencia del pipeline

Paso 1: configuración

La configuración aparece porque el contrato de datos exige registros seleccionables, límites temporales y rutas del dataset.

```
# src/semg_dashboard/config.py
PN_DIR = "cves/1.0.0/data/24h-electromyography"
SUBJECTS_URL = "https://physionet.org/files/cves/1.0.0/subjects.csv"

RECORDS = {
    "S0205 - ictus - NIHSS 10 / mRS 3": "s0205-05080904",
    "S0185 - ictus - NIHSS 0 / mRS 0": "s0185-05063014",
    "S0030 - control": "s0030-04051907",
}
```

La interfaz no debe contener rutas quemadas dentro de cada página.

Paso 2: carga de datos

La carga parcial es necesaria porque los registros pueden ser largos.

```
# src/semg_dashboard/data.py
@st.cache_data(show_spinner=False)
def load_emg_segment(record_name: str, start_s: int, duration_s: int):
    header = get_header(record_name)
    fs = float(header.fs)
    emg_idx = [i for i, name in enumerate(header.sig_name)
               if "emg" in name.lower()]

    sampfrom = int(start_s * fs)
    sampto = min(int((start_s + duration_s) * fs), int(header.sig_len))

    rec = wfdb.rdrecord(record_name, pn_dir=PN_DIR,
                        sampfrom=sampfrom, sampto=sampto,
                        channels=emg_idx)
    return rec.p_signal, float(rec.fs), rec.sig_name
```

Decisión teórica detrás de la carga parcial

No se carga todo el registro porque el dashboard requiere respuesta interactiva.

La decisión de ingeniería es:

registro completo $\rightarrow$ segmento temporal seleccionado

Ventaja: navegación rápida.

Costo: los indicadores describen solo el segmento seleccionado, no todo el día.

Paso 3: preprocesamiento

El módulo de procesamiento implementa la cadena definida antes.

```
# src/semg_dashboard/processing.py
def bandpass_semg(x, fs, low=20.0, high=450.0, order=4):
    x = interpolate_nan(x)
    high = min(high, 0.45 * fs)
    sos = signal.butter(order, [low, high],
                        btype="bandpass", fs=fs, output="sos")
    return signal.sosfiltfilt(sos, x)
```

El límite superior se ajusta a la frecuencia de muestreo para evitar filtros inválidos.

RMS y envolvente

```
# src/semg_dashboard/processing.py
def rms_envelope(x, fs, window_ms=250):
    n = max(3, int(window_ms / 1000.0 * fs))
    return np.sqrt(uniform_filter1d(np.asarray(x) ** 2,
                                     size=n, mode="nearest"))

def linear_envelope(x, fs, cutoff=5.0):
    rect = np.abs(x)
    sos = signal.butter(4, cutoff, btype="lowpass",
                       fs=fs, output="sos")
    return signal.sosfiltfilt(sos, rect)
```

El RMS responde a la pregunta de magnitud local. La envolvente facilita inspección visual.

Paso 4: umbral robusto de actividad

El umbral no debe depender de un valor arbitrario fijo.

```
# src/semg_dashboard/metrics.py
def robust_activity_threshold(rms):
    q20 = np.nanquantile(rms, 0.20)
    base = rms[rms <= q20]
    med = np.nanmedian(base)
    mad = 1.4826 * np.nanmedian(np.abs(base - med))
    return float(med + 3.0 * mad)
```

La hipótesis es que el percentil bajo aproxima periodos de menor actividad.

Paso 5: indicadores temporales

```
# src/semg_dashboard/metrics.py
def channel_metrics(raw, filtered, fs):
    rms = rms_envelope(filtered, fs, 250)
    env = linear_envelope(filtered, fs, 5.0)
    thr = robust_activity_threshold(rms)
    active = rms > thr
    starts, ends, durations = run_lengths(active, fs)
    minutes = len(filtered) / fs / 60.0

    return {
        "active_rms_median_uV": np.nanmedian(rms[active]),
        "active_time_pct": 100.0 * np.nanmean(active),
        "bursts_per_min": len(starts) / minutes,
        "median_burst_s": np.nanmedian(durations)
    }
```

Cada salida debe tener unidad e interpretación.

Paso 6: PSD y MDF

```
# src/semg_dashboard/metrics.py
def welch_psd(x, fs, n_seconds=4.0):
    x = interpolate_nan(x)
    nperseg = min(len(x), max(256, int(n_seconds * fs)))
    return signal.welch(x, fs=fs, nperseg=nperseg)

def median_frequency(x, fs, band=(20.0, 450.0)):
    f, pxx = welch_psd(x, fs)
    mask = (f >= band[0]) & (f <= min(band[1], 0.45 * fs))
    fb, pb = f[mask], pxx[mask]
    area = cumulative_trapezoid(pb, fb, initial=0.0)
    idx = np.searchsorted(area, 0.5 * area[-1])
    return float(fb[min(idx, len(fb)-1)])
```

Welch usa Fourier por segmentos y promedia periodogramas.

Paso 7: tabla de métricas

La tabla transforma indicadores por canal en una estructura visualizable.

```
# src/semg_dashboard/metrics.py
def metrics_table(raw_matrix, filtered_matrix, fs, names):
    processed = {}
    rows = []
    for j, name in enumerate(names):
        m = channel_metrics(raw_matrix[:, j], filtered_matrix[:, j], fs)
        processed[name] = m
        rows.append({
            "Canal": name,
            "RMS activo mediano [uV]": m["active_rms_median_uV"],
            "MDF [Hz]": m["mdf_hz"],
            "Tiempo activo [%]": m["active_time_pct"],
            "Episodios/min": m["bursts_per_min"]
        })
    return pd.DataFrame(rows), processed
```

Paso 8: figuras Plotly

Una figura Plotly debe responder una pregunta analítica.

```
# src/semg_dashboard/figures.py
def raw_filtered_figure(t, raw, filtered, channel):
    fig = go.Figure()
    fig.add_trace(go.Scatter(x=t, y=raw, name="Cruda"))
    fig.add_trace(go.Scatter(x=t, y=filtered, name="Filtrada"))
    fig.update_layout(
        title=f"{channel}: señal cruda y filtrada",
        xaxis_title="Tiempo [s]",
        yaxis_title="Amplitud [uV]",
        legend_orientation="h"
    )
    return fig
```

No basta con graficar: el eje, la unidad y el título son parte del análisis.

Figuras por propósito

Función	Propósito
<code>raw_filtered_figure()</code>	revisar artefactos y efecto del filtro
<code>rms_activity_figure()</code>	observar activación y umbral
<code>psd_figure()</code>	revisar distribución espectral
<code>metrics_bar_figure()</code>	comparar canales
<code>longitudinal_figure()</code>	observar cambio entre visitas

El nombre de la función debe revelar la pregunta que responde.

Paso 9: controles de usuario

```
# src/semg_dashboard/ui.py
def sidebar_controls():
    st.sidebar.header("Selección de datos")
    label = st.sidebar.selectbox("Registro", list(RECORDS))
    record_name = RECORDS[label]
    header = get_header(record_name)
    total_s = int(header.sig_len / header.fs)

    start_s = st.sidebar.number_input("Inicio [s]", min_value=0,
                                       max_value=max(0, total_s - 30),
                                       value=0, step=30)
    duration_s = st.sidebar.slider("Duración [s]", 30, 300, 120, 30)
    apply_notch = st.sidebar.checkbox("Aplicar notch 60 Hz", value=False)
    return record_name, int(start_s), int(duration_s), apply_notch
```

Los controles representan decisiones metodológicas visibles.

Paso 10: contexto clínico

```
# src/semg_dashboard/ui.py
def render_subject_context(record_name):
    subject_id = subject_from_record(record_name)
    subjects = load_subjects()
    subject = subjects.loc[subjects["subject_number"].eq(subject_id)]

    row = subject.iloc[0]
    cols = st.columns(5)
    cols[0].metric("Participante", subject_id)
    cols[1].metric("Grupo", row.get("group", "n/d"))
    cols[2].metric("Edad", row.get("age", "n/d"))
    cols[3].metric("NIHSS", row.get("NIHSS", "n/d"))
    cols[4].metric("mRS", row.get("MRS", "n/d"))
```

NIHSS y mRS contextualizan; no son salidas del algoritmo.

Paso 11: composición común

```
# src/semg_dashboard/ui.py
def compute_dashboard_context():
    record_name, start_s, duration_s, apply_notch = sidebar_controls()
    raw, fs, names = load_emg_segment(record_name, start_s, duration_s)

    filtered = np.column_stack([
        notch_60(bandpass_semg(raw[:, j], fs), fs)
        if apply_notch else bandpass_semg(raw[:, j], fs)
        for j in range(raw.shape[1])
    ])

    metrics_df, processed = metrics_table(raw, filtered, fs, list(names))
    t = np.arange(raw.shape[0]) / fs + start_s
    return record_name, fs, names, t, metrics_df, processed
```

Esta función evita repetir carga y procesamiento en cada página.

Paso 12: entrada principal

```
# app.py
from __future__ import annotations
import streamlit as st
from semg_dashboard.ui import render_home

st.set_page_config(page_title="sEMG post-ictus", layout="wide")
render_home()
```

app.py no procesa datos. Solo inicia la aplicación y presenta el propósito.

Página 1: inspección de señal

```
# pages/01_Inspeccion_sEMG.py
record_name, fs, names, t, metrics_df, processed = compute_dashboard_context()
render_subject_context(record_name)

selected = st.selectbox("Canal", list(names))
d = processed[selected]

st.plotly_chart(raw_filtered_figure(t, d["raw"], d["filtered"], selected),
                width="stretch")
st.plotly_chart(psd_figure(d["filtered"], fs, selected),
                width="stretch")
```

Pregunta que responde: ¿el segmento seleccionado es confiable para análisis?

Página 2: indicadores

```
# pages/02_Indicadores.py
st.subheader("Tabla de indicadores")
st.dataframe(metrics_df, width="stretch", hide_index=True)

selected = st.selectbox("Canal para inspección", list(names))
d = processed[selected]
st.plotly_chart(rms_activity_figure(t, d["rms"], d["envelope"],
                                   d["threshold"], selected),
                width="stretch")
```

Pregunta que responde: ¿qué canales muestran actividad y cómo se cuantifica?

Página 3: seguimiento

```
# pages/03_Seguimiento.py
st.warning(
    "CVES es transversal. Esta visualización es una estructura didáctica, "
    "no una trayectoria clínica real."
)

long_df = longitudinal_demo(metrics_df)
st.plotly_chart(longitudinal_figure(long_df), width="stretch")
st.dataframe(long_df, width="stretch", hide_index=True)
```

Pregunta que responde: ¿cómo se organizaría una visualización longitudinal si existieran visitas repetidas?

Sección 8

Parte VIII - Evaluación crítica del dashboard

Qué debe observar el estudiante

Al ejecutar el dashboard, el estudiante debe inspeccionar:

- si los canales EMG aparecen correctamente;
- si la señal cruda tiene artefactos dominantes;
- si el filtro altera de forma razonable la señal;
- si el umbral de actividad es defendible;
- si la PSD está concentrada en bandas fisiológicamente plausibles;
- si los indicadores tienen unidad y sentido clínico.

Errores frecuentes

Errores técnicos comunes

- usar FFT cruda como si fuera PSD estable;
- aplicar notch sin justificar presencia de ruido de red;
- comparar RMS entre sesiones sin normalización ni protocolo;
- inferir músculo o lateralidad desde nombres genéricos de canal;
- interpretar Stroke Side sin revisar definición del dataset;
- presentar el dashboard como herramienta diagnóstica.

Preguntas de verificación

- 1 ¿Qué pregunta clínica responde cada pestaña del dashboard?
- 2 ¿Qué indicador depende más del umbral de actividad?
- 3 ¿Por qué MDF se estima desde PSD y no desde la señal cruda?
- 4 ¿Qué condiciones permitirían interpretar ΔRMS_{rel} longitudinalmente?
- 5 ¿Qué advertencias debe ver un médico antes de interpretar el panel de seguimiento?

Actividad práctica

Modificar el dashboard para incluir:

- selector de ventana RMS: 100, 250 y 500 ms;
- tarjeta de advertencia si baja frecuencia supera un umbral;
- exportación CSV de indicadores;
- visualización comparativa entre dos segmentos temporales;
- texto automático que explique si el segmento es apto para análisis.

La modificación debe respetar la arquitectura multi-script.

Crterios de calidad

Criterio	Evidencia esperada
Coherencia biomédica	indicadores conectados con fisiología
Reproducibilidad	parámetros visibles y funciones separadas
Calidad de señal	advertencias explícitas
Visualización	ejes, unidades, títulos e interacción útil
Modularidad	páginas y módulos separados
Interpretación	conclusiones prudentes y no diagnósticas

Cierre

Un dashboard biomédico riguroso no se diseña desde la librería gráfica.

Se diseña desde una cadena metodológica explícita:

pregunta clínica → contrato de datos
→ procesamiento
→ indicadores
→ visualización
→ interpretación.

Streamlit y Plotly son herramientas. La validez del dashboard depende del pipeline conceptual que las gobierna.

Archivos entregados

El paquete incluye:

```
tutorial_streamlit_plotly_semg.qmd  
beamer_header.tex  
referencias.bib  
assets/*.pdf  
dashboard_semg_ictus/  
  app.py  
  pages/  
  src/semg_dashboard/  
  pyproject.toml  
  README.md
```

Ejecución

Para renderizar la presentación:

```
quarto render tutorial_streamlit_plotly_semg.qmd
```

Para ejecutar el dashboard:

```
cd dashboard_semg_ictus  
uv sync  
uv run streamlit run app.py
```

Referencias

Novak, Vera. 2018. *Cerebral Vasoregulation in Elderly with Stroke*. PhysioNet, version 1.0.0.
<https://doi.org/10.13026/C2DW96>.